

CarZaar

Car bidding marketplace - User app & Admin app

Scope	user module (buyer and seller mode) and admin module,
Method	Manual static code review of the source - White Box Testing Static and Dynamic Application Security Testing.
Date	18 July 2026

1. Findings overview:

Finding	Severity	Category
No server-side authorization as every rule is enforced in the client side	Critical	Broken Access Control
Passwords stored and compared in plain text	Critical	Cryptographic Failures
Hard coded admin credentials that are checked only on the device.	Critical	Identification and Auth Failures
Admin and moderation actions performed with the public anon key leading to privilege escalation	Critical	Broken Access Control
Connects balance can be created, edited and spent by the client as there are no real payment checks lead to race conditions	Critical	Business Logic
Password reset only needs e-mail and phone number	High	Identification and Auth Failures
Whole user row including passwords returned to other users.	High	Excessive Data Exposure
IDOR as record is changed by id only	High	Broken Object Level Authorization
Mass assignment as client chooses privileged fields like (verified, status, featured, bid status)	High	Broken Object Property Level Authorization
Bid and listing business rules enforced only in the UI	Medium	Insecure Design
User enumeration and verbose error messages	Medium	Information Disclosure
No brute force protection	Medium	Identification and Auth Failures
Blocked accounts keep working until they log in again	Medium	Broken Access Control
Unrestricted base64 image upload stored in database rows	Medium	Unrestricted Resource Consumption
Supabase URL and anon key hard-coded and about to be published (fixed)	Medium	Security Misconfiguration
Release build signed with debug key like default package id hence no obfuscation	Low	Security Misconfiguration
WhatsApp link built from unvalidated phone text so (partially mitigated)	Low	Injection
Admin actions are not attributable or logged and admin "logout" is only navigation	Low	Security Logging and Monitoring Failures

2. Detailed findings:

Broken Access Control:

LOCATION:

```
user_setup/lib/Services  
Carzaar admin/Carzaar/lib  
user_setup/lib/main.dart
```

DESCRIPTION

Both apps talk to Supabase directly from the device using the public **anon** key. No Supabase Auth session (JWT with a user id) is used, so the database cannot tell *who* is calling. Unless Row Level Security (RLS) policies are written that do not depend on the client, anybody who holds the anon key can read and write every table directly through the REST API, bypassing all checks in the UI.

EVIDENCE

```
await Supabase.initialize(url: 'https://lweqtpuwumggheksokvo.supabase.co',  
    anonKey: 'eyJhbGciOi...3owkdo')
```

IMPACT

Full read / write access to all users, cars, bids, wallets, reports and transactions by anyone, without installing the app. Every other finding below becomes trivially exploitable.

Cryptographic Failures:

LOCATION:

```
user_setup/lib/Services/auth_service.dart  
user_setup/lib/Models/user_model.dart  
user_setup/lib/Authentication/SignUp.dart
```

DESCRIPTION

The password typed at sign-up is inserted unchanged into `user.p a s s w o r d` and login simply asks the database for a row where the stored value equals the typed value. Anyone who can read the table (or a database backup log) sees every password. Because many people reuse passwords this also endangers their other accounts.

EVIDENCE

```
.from('users').select(  
    )  
    .eq('email', email)  
    .update({'password': newPassword})
```

IMPACT

Mass credential disclosure; account takeover of every user; credential-stuffing against other sites.

Identification & Auth Failures:

LOCATION

```
Carzaar admin/Carzaar/lib/admin_utils.dart  
Carzaar admin/Carzaar/lib/Authentication/AdminLogin.dart  
Carzaar admin/Carzaar/README_ADMIN_SUPABASE.txt
```

DESCRIPTION

The admin e-mail and password are constants in the source code. The comparison happens inside the app so there is no real session: the admin screens simply call the database with the same anon key every user has. Anyone who decompiles the APK / web bundle or reads source code gets admin access.

EVIDENCE

```
const adminEmail = 'admin@carzaar.com';
const adminPassword = 'admin123';
if (email.text.trim() == adminEmail && password.text.trim() == adminPassword)
```

IMPACT

Complete administrative control of the platform (block users, hide listings, close complaints) for anyone.

Broken Function Level Authorization:

LOCATION

```
Carzaar admin/Carzaar/lib/UserManagement/banUser.dart
Carzaar/lib/CarListingMonitoring/approveListing.dart
Carzaar admin/Carzaar/lib/BidMonitoring/removeBid.dart
Carzaar admin/Carzaar/lib/ReportComplaint/resolveReport.dart
```

DESCRIPTION

Blocking users, activating / blocking listings, rejecting bids and resolving complaints are plain table updates that any caller of the REST API can issue. The "admin only" nature exists only because the admin screens

EVEDENCE

```
await supabase.from('users').update({'status': next}).eq('id', widget.userId);
await supabase.from('cars').update({'status': 'active'}).eq('id', widget.carId)
```

IMPACT

Moderation can be bypassed or abused: banned users reinstate themselves so malicious users ban competitors; complaints can be erased.

Insecure Design:

LOCATION

```
user_setup/lib/Services/connects_service.dart
user_setup/lib/Connects/buyConnects.dart
```

DESCRIPTION

- (a) "Buying" connects only checks that the demo card fields are *not empty*, then the app itself writes the new balance. There is no payment provider, receipt or server verification.
- (b) Balance changes are a read-modify-write (`getBalance` then `update balance = old + new`) executed from the device, so two parallel requests can overwrite each other and a caller can set any value directly.

EVIDENCE

```
final currentBalance = await getBalance(userId);
final newBalance = currentBalance + connects;
await _client.from('connects_wallet').update({'balance': newBalance, ...}).eq('user_id', userId);
```

IMPACT

Unlimited free connects; revenue model bypassed; inconsistent ledger. (Acceptable only if clearly labelled as a demo.

Identification & Auth Failures

LOCATION

```
user_setup/lib/Services/auth_service.dart
user_setup/lib/Seller/receivedBids.dart
user_setup/lib/Services/bid_service.dart
```

DESCRIPTION

A password is replaced when the caller supplies a matching e-mail and phone. Both are weak "secrets": the phone number is displayed to sellers in *Received Bids* and returned in every `select()` on the users table and e-mail addresses are easy to learn. No e-mail / SMS one-time code or ownership proof is required.

EVIDENCE (code)

```
final user = await _client.from('users').select()
    .eq('email', email).eq('phone', phone).maybeSingle();
...
await _client.from('users').update({'password': newPassword})
    .eq('email', email).eq('phone', phone);
```

IMPACT

Takeover of any account whose phone number has been seen by another user; access to its wallet, listings and bids.

Sensitive Data Exposure

LOCATION

```
user_setup/lib/Services/bid_service.dart
user_setup/lib/Services/report_service.dart
user_setup/lib/Buyer/myBids.dart
```

DESCRIPTION

`getUserById`, `getSellerDetails` and others run, `select()` on `users` and hand the full row to the UI. The UI only shows the name/phone but the password, e-mail and profile image of the other party are in the response and in memory. The seller's phone is also loaded before the buyer has paid to unlock it.

EVIDENCE

```
final response = await _client.from('users').select().eq('id', userId).maybeSingle();
final sellerPhone = seller?['phone'] ?? "";
```

IMPACT

Disclosure of other users' passwords and personal data; bypass of the paid "Unlock Contact" feature (the phone number is delivered for free).

IDOR:

LOCATION

```
user_setup/lib/Services/bid_service.dart
user_setup/lib/Services/car_service.dart
user_setup/lib/Services/connects_service.dart
```

DESCRIPTION

Update or delete filter only by the record id and never by the owner, any logged-in caller who knows or guesses an id can edit, withdraw, accept, reject, cancel or delete somebody else bid or car.

EVIDENCE

```
.update({'bid_status': 'accepted'}).eq('id', bidId);
.update({'status': 'removed'}).eq('id', carId);
.from('cars').update({'is_featured': true}).eq('id', carId);
```

IMPACT

Sabotage of competitors' bids and listings, forced "accepted" deals, fake sales.

Broken Object Property Level Authorization

LOCATION

```
user_setup/lib/Models/user_model.dart
user_setup/lib/Models/car_model.dart
user_setup/lib/Models/bid_model.dart
user_setup/lib/Services/auth_service.dart
```

DESCRIPTION

The models serialise *every* column, including `is_verified`, `status`, `is_featured` and `bid_status` and the whole object is inserted. Because nothing server-side restricts these columns, a caller can create a pre-verified seller, a featured listing without paying 10 connects or a bid that is already "accepted".

EVIDENCE

```
Map<String, dynamic> toJson() => {'name': name, ..., 'is_verified': isVerified, 'status': status};

.from('cars').insert(car.toJson());
```

IMPACT

Free premium features (revenue loss), fake "Verified Seller" badge (trust abuse), forged bid states.

Insecure Design:

LOCATION

```
user_setup/lib/Buyer/carDetails.dart
user_setup/lib/Bidding/bidCar.dart
user_setup/lib/Services/bid_service.dart
```

DESCRIPTION

- 1) "You cannot bid on your own car" and "bid must exceed the starting price" exist only in Flutter widgets.
- 2) `updateBid` changes the amount and silently resets the status to *pending* for any bid, including an *accepted* one, so a buyer can lower an accepted offer after the seller agreed.
- 3) `acceptBid` checks for an existing accepted bid and then updates in a second call, so two bids can be accepted at once.

EVIDENCE

```
.update({'bid_amount': newAmount, 'bid_status': 'pending'}).eq('id', bidId);

final acceptedBid = await .... eq('bid_status','accepted').maybeSingle();
await _client.from('bids').update({'bid_status': 'accepted'}).eq('id', bidId);
```

IMPACT

Price manipulation after agreement, self-bidding to inflate prices, double acceptance.

Information Disclosure:

LOCATION

```
user_setup/lib/Services/auth_service.dart
user_setup/lib/Services/bid_service.dart
```

DESCRIPTION

Sign up says "Email already registered", the reset screen says "Email and phone do not match" (separate checks reveal which part was right) and many screens show the raw exception text, which can contain table names, column names and SQL error details.

EVIDENCE

```
return "Email already registered";  
return "Verification failed. Email and phone do not match.";  
return "Signup failed: $e";
```

IMPACT

Helps attackers build target lists and learn the schema.